

Logical Thinking

Lecture 1: Introduction to Logic and Logical Thinking

1. Logic

What is logic?

Logic is the systematic study of correct reasoning.

It provides rules that help us determine whether a conclusion follows correctly from given information.

In simple words, logic teaches us how to think in a structured way rather than making random guesses.

Real-life example

Imagine the following statements:

- All humans need oxygen.
- Ali is a human.
- Therefore, Ali needs oxygen.

Let's examine the reasoning:

1. We have a general statement: all humans need oxygen.
2. We know Ali belongs to the category of humans.
3. We conclude that Ali needs oxygen.

Computers use logical principles in many operations.

Types of logic

- Formal logic: Studies reasoning through precise rules and symbols.
- Informal logic: Examines reasoning in everyday language and arguments.
- Mathematical logic: Applies formal reasoning to mathematical structures and proofs.
- Propositional logic: Studies statements that are either true or false and how they combine.

- Predicate logic: Studies statements involving objects, properties, and relationships.

Remember: Logic is the foundation; programming is one of its practical applications.

2. Logical Thinking

What is logical thinking?

Logical thinking is the ability to analyze information, identify relationships, evaluate evidence, and reach a reasonable conclusion using structured reasoning.

In simple words, logical thinking means solving a problem step by step instead of guessing or making decisions based only on emotions.

Logic gives us the rules of reasoning, while logical thinking is the process of applying those rules.

Example from everyday life

Imagine you wake up in the morning and notice that the road outside is wet.

You might think:

- The road is wet.
- Rain can make roads wet.
- Perhaps it rained during the night.

Notice that your conclusion is only a possibility. Someone might have washed the road instead.

This is an example of logical thinking, but it also teaches us an important lesson: a reasonable conclusion is not always a certain conclusion.

The process of logical thinking

1

Identify the problem

Understand exactly what needs to be solved.

2

Collect information

Gather relevant facts and available evidence.

3

Analyze information

Identify patterns, relationships and possible explanations.

4

Evaluate possibilities

Check assumptions and eliminate incorrect possibilities.

5

Reach a conclusion

Choose a conclusion supported by the available evidence.

Logical thinking in computer science

Suppose you are developing a program that determines whether a student has passed an examination.

Your university uses the following rule:

- Marks of 50 or above: Pass.
- Marks below 50: Fail.

You can think through the problem logically:

1. Receive the student's marks.
2. Compare the marks with 50.
3. If marks are greater than or equal to 50, display Pass.
4. Otherwise, display Fail.

Logical thinking vs. ordinary thinking

Ordinary thinking

May rely on assumptions

May jump to conclusions

Logical thinking

Examines evidence

Follows a structured process

May overlook alternatives | Considers different possibilities

Can be driven by emotions | Focuses on reasoning and evidence

Ordinary thinking is not necessarily wrong. Logical thinking simply provides a more systematic way to solve problems.

3. Algorithm and Logic

Algorithm and logic are closely connected. Logic helps us determine what steps are correct, while an algorithm organizes those steps into a sequence to solve a particular problem.

3.1 What is an algorithm?

An algorithm is a finite, ordered sequence of clear instructions designed to solve a problem or perform a task.

In simple words, an algorithm is a step-by-step procedure for reaching a desired result.

Real-life example: Making tea

Imagine you want to make a cup of tea.

→Start

→Boil water

→Add tea leaves

→Add milk and sugar

→Cook for a few minutes

→Pour tea into a cup

→End: Tea is ready

This is a simple algorithm because the instructions are arranged in a sequence and produce a desired result.

However, a real algorithm should also account for important details, such as how much water to use and how long to cook the tea.

3.2 Characteristics of an algorithm

A properly designed algorithm generally has these characteristics:

Characteristic	Explanation
Input	Accepts zero or more inputs
Output	Produces at least one result
Definiteness	Every instruction must be clear and unambiguous
Finiteness	Must terminate after a finite number of steps
Effectiveness	Each step must be sufficiently simple and executable

Let's understand them with an example.

Suppose we want to calculate the sum of two numbers.

- Input: Two numbers, A and B.
- Process: Add A and B.
- Output: Their sum.
- Definiteness: The addition operation is clearly specified.
- Finiteness: The process ends after performing the addition.

3.3 What is the relationship between algorithm and logic?

Consider a problem:

Find the largest number among three numbers.

We need logic to determine how to compare the numbers correctly.

Suppose:

- $A = 25$
- $B = 40$
- $C = 15$

Our logical reasoning is:

1. If A is greater than or equal to B and C, A is the largest.
2. Otherwise, if B is greater than or equal to A and C, B is the largest.
3. Otherwise, C is the largest.

The algorithm is the ordered set of these comparison steps.

3.4 Algorithm vs. logic

Logic	Algorithm
Rules and principles of correct reasoning	A specific procedure for solving a problem
Helps determine which decisions are correct	Organizes instructions into steps
Used to evaluate conclusions	Used to execute a solution
Can be expressed mathematically or verbally	Can be expressed using pseudocode, flowcharts or code

Think of it this way:

- Logic is the reasoning behind your solution.
- An algorithm is the structured plan that applies that reasoning.
- A program is an implementation of that plan in a programming language.

Remember this relationship :

Problem

Logical reasoning

Algorithm

Computer program

Result

3.5 Why algorithms matter in computer science

Algorithms are used in nearly every area of computer science:

- Searching: Finding a particular item in a large dataset.
- Sorting: Arranging data in a specific order.
- Artificial intelligence: Finding solutions and making decisions.
- Cybersecurity: Applying cryptographic procedures and security checks.
- Networking: Routing data through networks.
- Database systems: Retrieving and processing records efficiently.

An algorithm can solve a problem correctly but still be inefficient. This is why computer scientists also study time and space complexity.

4. Arguments

In logic, an argument does not mean a fight or a disagreement. It means a collection of statements in which some statements provide reasons or evidence to support another statement.

Definition:

An argument is a set of statements consisting of one or more premises(Land and the buildings on it) that are intended to support a conclusion.

Arguments are essential in mathematics, computer science, philosophy, and everyday decision-making.

4.1 Components of an argument

An argument has two main components:

- Premises: Statements that provide reasons or evidence.
- Conclusion: The statement that the premises are intended to support.

Let's understand with a simple example.

Example:

- Premise 1: All computers need electricity to operate.
- Premise 2: A laptop is a computer.
- Conclusion: Therefore, a laptop needs electricity to operate.

ARGUMENT STRUCTURE

Premise 1

All computers need electricity to operate.

Premise 2

A laptop is a computer.

Therefore

Conclusion

A laptop needs electricity to operate.

The premises provide the reasoning, and the conclusion follows from them.

4.2 How to identify premises and conclusions

In everyday language, certain words often indicate premises or conclusions.

Premise indicators	Conclusion indicators
Because	Therefore
Since	Thus
Given that	Hence
As	Consequently
Due to the fact that	It follows that

Example:

"Since all humans are mortal and Ahmed is a human, therefore Ahmed is mortal."

- Premise 1: All humans are mortal.
- Premise 2: Ahmed is a human.
- Conclusion: Ahmed is mortal.

Remember that these indicator words are clues, not absolute rules. A statement can be a premise or conclusion even without these words.

4.3 Types of arguments

There are two major types of reasoning commonly studied in introductory logic.

A. Deductive arguments

A deductive argument attempts to establish its conclusion with necessity: if the premises are true and the reasoning is valid, the conclusion must be true.

Example:

- All even numbers are divisible by 2.
- 8 is an even number.
- Therefore, 8 is divisible by 2.

The conclusion necessarily follows from the premises.

Deductive reasoning is widely used in mathematics, formal verification, and program correctness.

B. Inductive arguments

An inductive argument uses observations or evidence to support a conclusion that is probable rather than guaranteed.

Example:

- The sun rose yesterday.
- The sun rose the day before yesterday.
- The sun has risen every day in our observations.
- Therefore, the sun will probably rise tomorrow.

The evidence supports the conclusion, but the conclusion does not follow with deductive certainty.

Inductive reasoning is important in data science, machine learning, scientific research, and prediction.

Deductive vs. inductive reasoning

Deductive	Inductive
Seeks necessary conclusions	Supports probable conclusions
Evaluated using validity and soundness	Often evaluated using strength and cogency
Used in mathematical proofs	Used in predictions and generalizations
True premises in a valid argument guarantee a true conclusion	True premises do not guarantee the conclusion

4.4 Arguments in computer science

Let's take a programming example.

Suppose you have a program that checks whether a user is allowed to enter a system.

The rules are:

- A user must be logged in.
- A user must have administrator privileges.
- If both conditions are true, access is granted.

Here, the program implements a logical rule. The conditions are premises, and the access decision is the resulting conclusion.

In formal reasoning, we can also represent the argument as:

- If P and Q, then R.
- P is true.
- Q is true.
- Therefore, R is true.

This is a valid deductive argument.

4.5 An argument is not the same as an explanation

This is a subtle but important distinction.

Argument:

- All metals conduct electricity under suitable conditions.
- Copper is a metal.
- Therefore, copper conducts electricity under suitable conditions.

The premises are offered to establish a conclusion.

Explanation:

- The copper wire conducts electricity because copper contains mobile electrons that allow electric charge to flow.

The second statement explains why something happens rather than attempting to prove that it happens.

An explanation may contain reasoning and facts, but its purpose differs from that of an argument.

Important points to remember

- An argument is a structure of reasoning, not necessarily a disagreement.
- Premises are supporting statements.
- Conclusions are the claims being supported.
- Deductive arguments aim for certainty when valid.
- Inductive arguments support conclusions with varying degrees of probability.
- A strong argument requires more than convincing language; its reasoning and supporting premises must be examined.

5. Truth, Validity, and Soundness

5.1 Truth

Truth is a property of a statement that correctly represents reality or is true under a given interpretation.

In classical logic, a proposition has one of two truth values:

- True (T): The statement is true.
- False (F): The statement is false.

Examples

Statement	Truth value
$2 + 2 = 4$	True
10 is less than 5	False
Islamabad is the capital of Pakistan	True
7 is an even number	False

Notice that truth is a property of individual statements, not the structure of an argument.

5.2 Validity

Validity is a property of an argument. An argument is valid if it is impossible for all its premises to be true while its conclusion is false.

This definition is extremely important.

Validity does not ask whether the premises are actually true in reality. It asks whether the conclusion follows correctly from the premises.

Example 1: A valid argument

- Premise 1: All birds are animals.
- Premise 2: A sparrow is a bird.
- Conclusion: A sparrow is an animal.

The conclusion follows necessarily from the premises.

Example 2: A valid argument with a false premise

- Premise 1: All fish can fly.
- Premise 2: A shark is a fish.
- Conclusion: A shark can fly.

The first premise is false, and the conclusion is also false. However, the argument's structure is valid.

Why?

If all fish could fly, and a shark were a fish, then the conclusion that a shark could fly would follow.

This shows that validity is about the logical relationship between premises and conclusions, not whether the statements describe the actual world.

Remember this rule

A valid argument cannot have all true premises and a false conclusion.

However, a valid argument can have false premises, and it can have a false conclusion.

5.3 Soundness

Soundness is a property of a deductive argument that is both valid and has all true premises.

For an argument to be sound, it must satisfy two conditions:

1. Its logical structure must be valid.
2. Every premise must actually be true.

If both conditions are satisfied, its conclusion must also be true.

Example of a sound argument

- Premise 1: All squares have four sides.
- Premise 2: Figure A is a square.
- Conclusion: Figure A has four sides.

This argument is sound because the premises are true and the reasoning is valid.

Example of an unsound argument

- Premise 1: All computers are laptops.
- Premise 2: A desktop is a computer.
- Conclusion: A desktop is a laptop.

The argument has a valid logical structure, but its first premise is false. Therefore, the argument is unsound.

5.4 Difference between truth, validity, and soundness

This is the most important comparison in this lecture.

Concept	Applies to	Main question
Truth	Individual statements	Is the statement true or false?
Validity	Arguments	Does the conclusion follow logically from the premises?
Soundness	Deductive arguments	Is the argument valid, and are all premises true?

Let's visualize how validity and soundness are related.

All deductive arguments

Arguments that attempt to establish conclusions through deduction

Valid arguments

Correct logical structure

Sound arguments

Valid arguments with all true premises

Every sound argument is valid, but not every valid argument is sound.

Also, an argument can be invalid even when all its premises and its conclusion happen to be true.

For example:

- Premise 1: 2 is an even number.
- Premise 2: 4 is an even number.
- Conclusion: 6 is an even number.

All three statements are true. However, the premises do not logically guarantee the conclusion. This particular argument is invalid.

5.5 Truth tables: connection to computer science

In propositional logic, truth tables help us determine the truth value of compound statements.

Consider the logical AND operator, represented by $\wedge$ and AND .

The expression $P \wedge Q$ and $P \text{ AND } Q$ is true only when both P and Q are true.

P	Q	P AND Q
---	---	---------

T	T	T
---	---	---

T	F	F
---	---	---

F	T	F
---	---	---

F	F	F
---	---	---

Now consider the implication $P \rightarrow Q$ and $P \rightarrow Q$.

P	Q	$P \rightarrow Q$
---	---	-------------------

T	T	T
---	---	---

T	F	F
---	---	---

F	T	T
---	---	---

F	F	T
---	---	---

An implication is false only when its antecedent is true and its consequent is false.

These tables are useful for understanding conditional statements, designing digital circuits, and checking the validity of logical arguments.

Final lesson summary

- Truth evaluates statements.
- Validity evaluates the logical structure of arguments.
- Soundness requires a valid argument with true premises.
- Truth tables help analyze compound statements and logical relationships.
- In computer science, these concepts help establish the correctness of algorithms, programs, and mathematical proofs.

6. Fallacies

(IN SECOND PDF FORMAT)

Lecture 7: Diagrammatic Logic

7.1 What is diagrammatic logic?

Diagrammatic logic is the use of visual representations, such as diagrams, shapes, and graphical structures, to represent and analyze logical relationships between concepts or statements.

In simple words, it means using pictures, circles, boxes, or other visual structures to understand and solve reasoning problems.

Diagrammatic logic is useful in:

- Mathematical reasoning
- Computer science
- Artificial intelligence
- Database design
- Set theory
- Problem-solving and analytical reasoning

7.2 Venn diagrams

A Venn diagram is one of the most common tools used in diagrammatic logic.

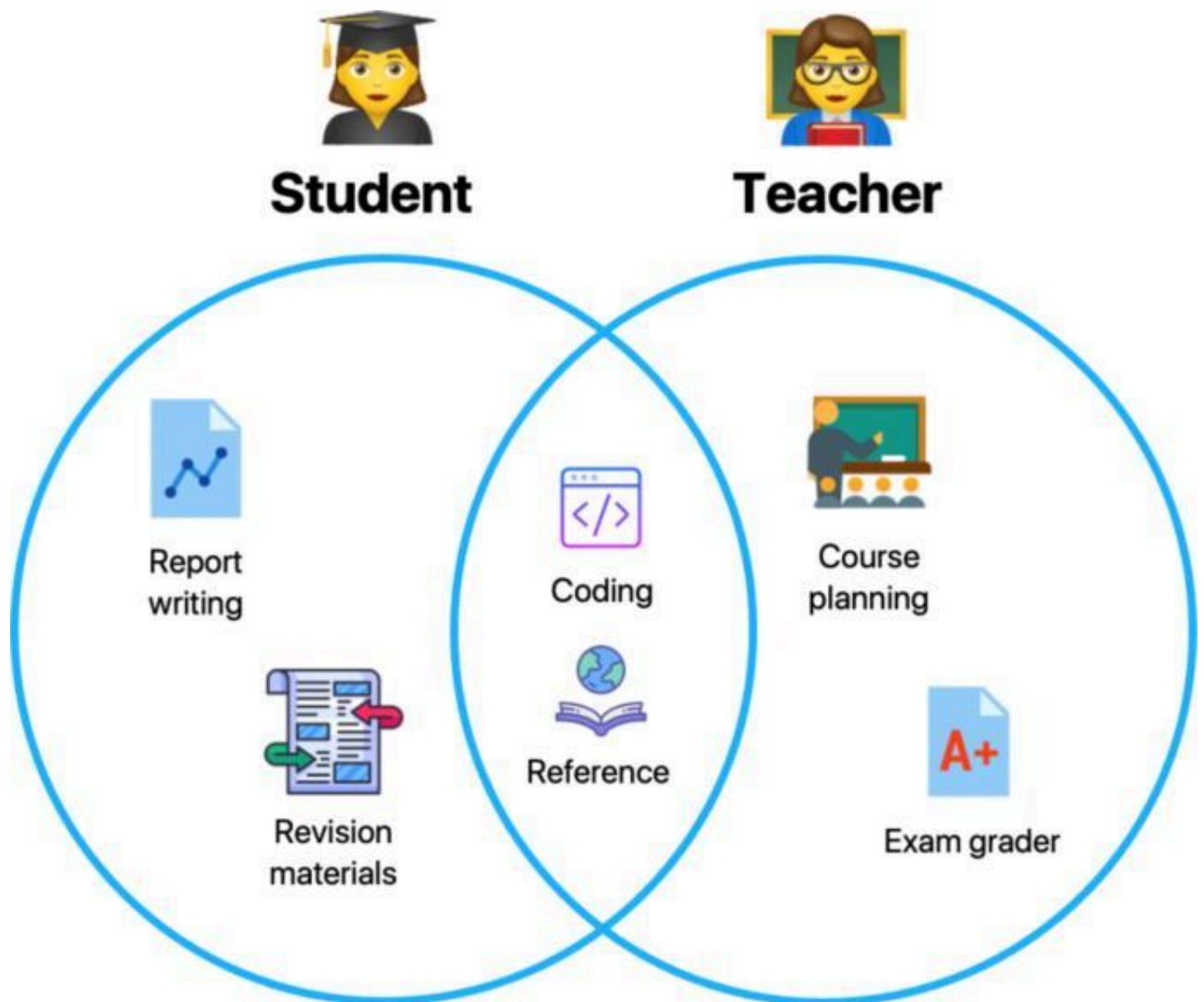
It uses overlapping circles to represent sets and their relationships.

A set is a collection of distinct objects.

For example:

- Set A: All students
- Set B: All programmers

Some students may be programmers, while others may not be.



Two overlapping sets. The shared region represents students who are also programmers.

Understanding the regions

- A only: Objects belonging to set A but not B.
- B only: Objects belonging to set B but not A.
- Intersection: Objects belonging to both A and B.
- Outside both: Objects belonging to neither set, within the defined universal set.

Mathematically, the intersection is written as:

$$A \cap B \text{ or } B \cap A$$

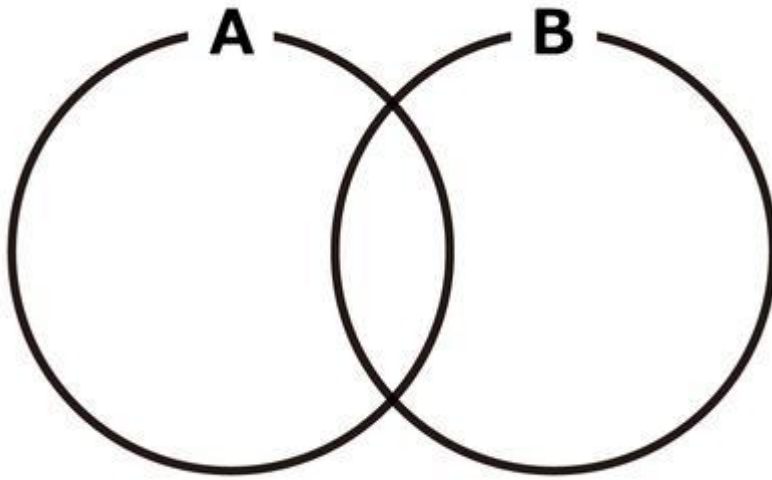
The union of two sets is written as:

$$A \cup B \text{ or } B \cup A$$

It includes all objects that belong to A, B, or both.

7.3 Types of relationships in Venn diagrams

Let's examine four important relationships.



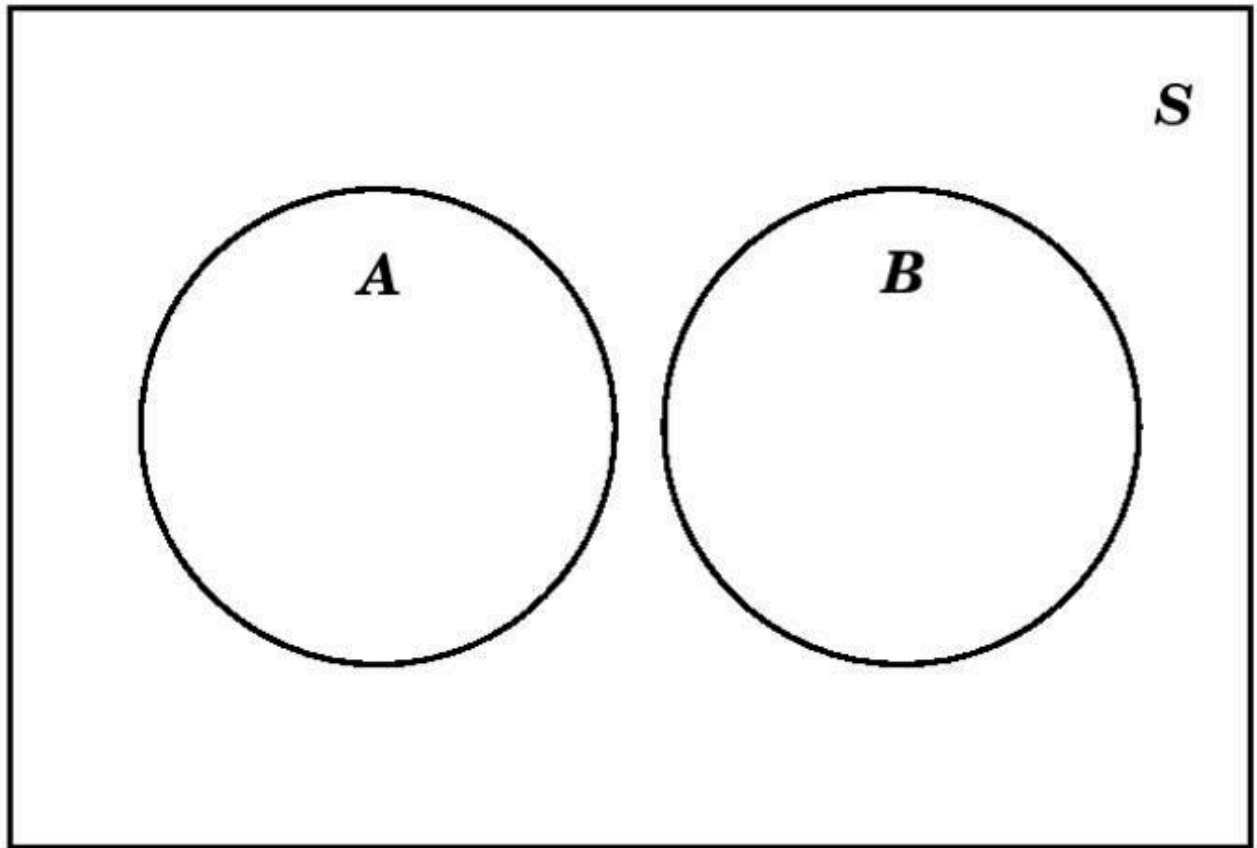
1. Overlapping sets

Two sets share some elements, but each may also contain elements not belonging to the other.

Example:

- A = Students who study Python
- B = Students who study Java

Some students study both languages.



2. Disjoint sets

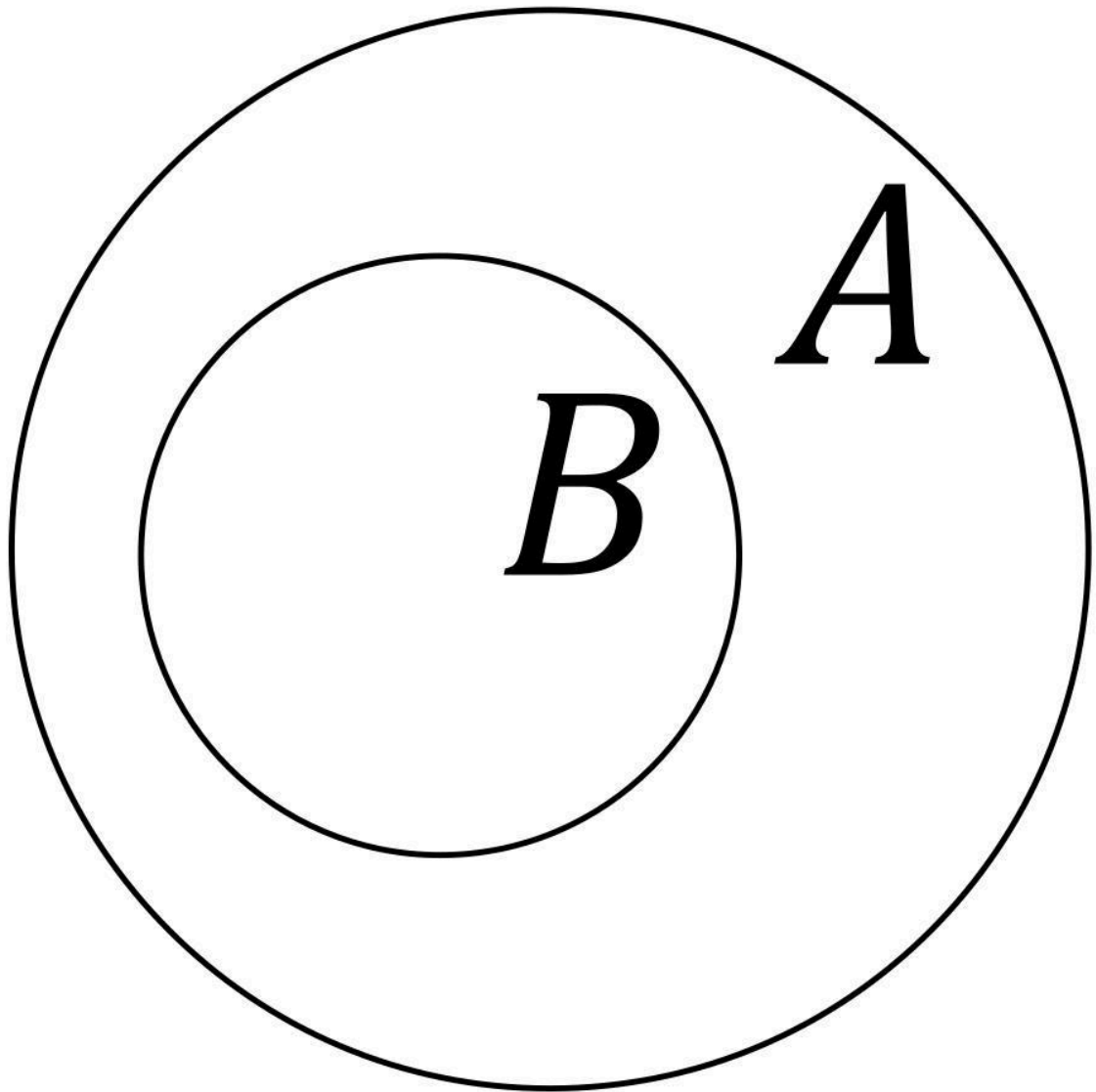
Two sets are disjoint if they have no elements in common.

$$A \cap B = \emptyset \quad A \setminus B = \emptyset \quad A \cap B = \emptyset$$

Example:

- A = Even integers
- B = Odd integers

No integer is both even and odd.



3. Subset relationship

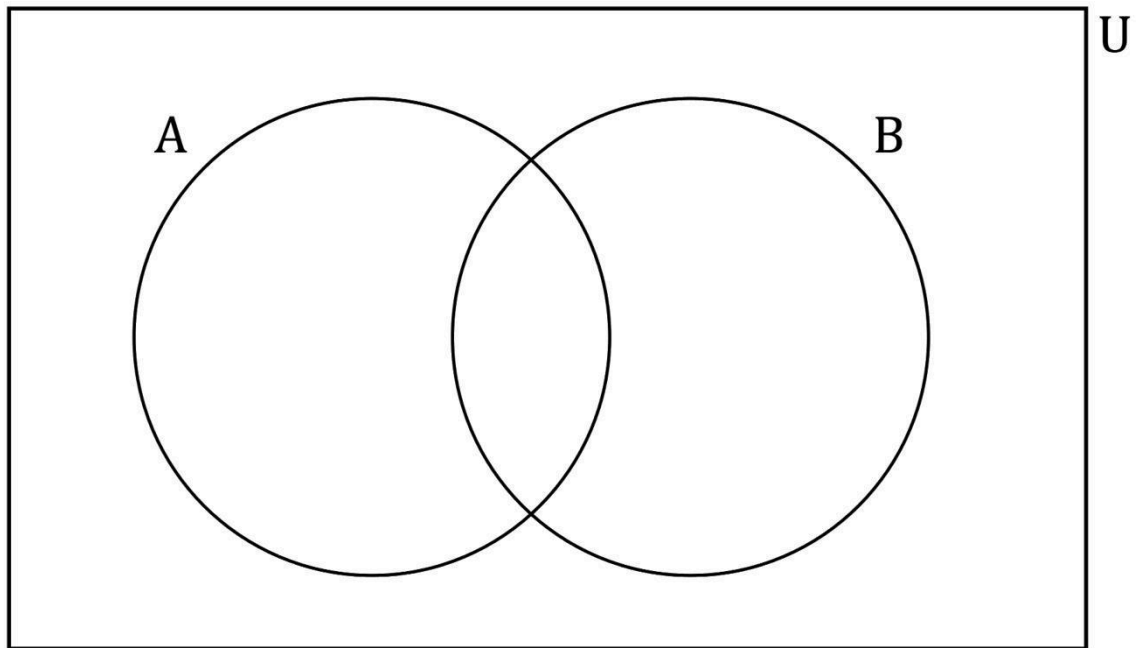
Set B is a subset of A if every element of B is also an element of A.

$$B \subseteq A \quad \text{or} \quad A \supseteq B$$

Example:

- A = All programming languages
- B = Object-oriented programming languages

Assuming the categories are defined this way, every member of B belongs to A.



4. Universal set

A universal set contains all the elements being considered in a particular problem.

Example:

- U = All students in a university
- A = Students enrolled in computer science
- B = Students enrolled in mathematics

Both A and B are subsets of U.

7.4 Set operations used in diagrammatic logic

Set operations are important because they help us express relationships precisely.

Operation	Symbol	Meaning
Union	$A \cup B$ $B \cup A$	Elements in A or B or both
Intersection	$A \cap B$ $B \cap A$	Elements common to A and B
Difference	$A - B$ $B - A$	Elements in A but not B
Complement	A' A^c	Elements outside A but within U

Empty set $\emptyset$ A set with no elements

Let's take a simple example:

$A = \{1, 2, 3, 4\}$

$B = \{3, 4, 5, 6\}$

Then:

- $A \cup B = \{1, 2, 3, 4, 5, 6\}$
- $A \cap B = \{3, 4\}$
- $A - B = \{1, 2\}$
- $B - A = \{5, 6\}$

These operations are useful in database queries, filtering, and reasoning about collections of data.

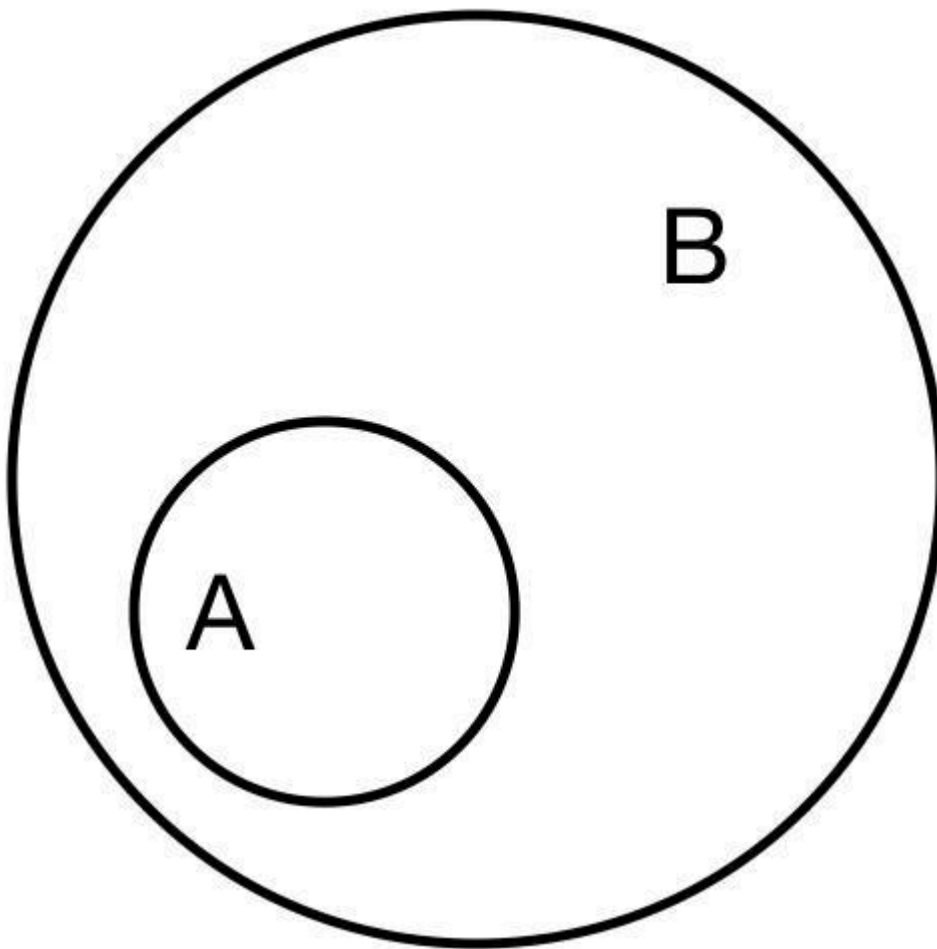
7.5 Using Venn diagrams to analyze arguments

Now let's connect diagrams with formal logic.

Consider these statements:

- All programmers are problem-solvers.
- Ali is a programmer.
- Therefore, Ali is a problem-solver.

We can represent the categories as nested sets.



All programmers are contained within the set of problem-solvers.

Because the set of programmers is entirely inside the set of problem-solvers, anyone who belongs to the programmers set must also belong to the problem-solvers set.

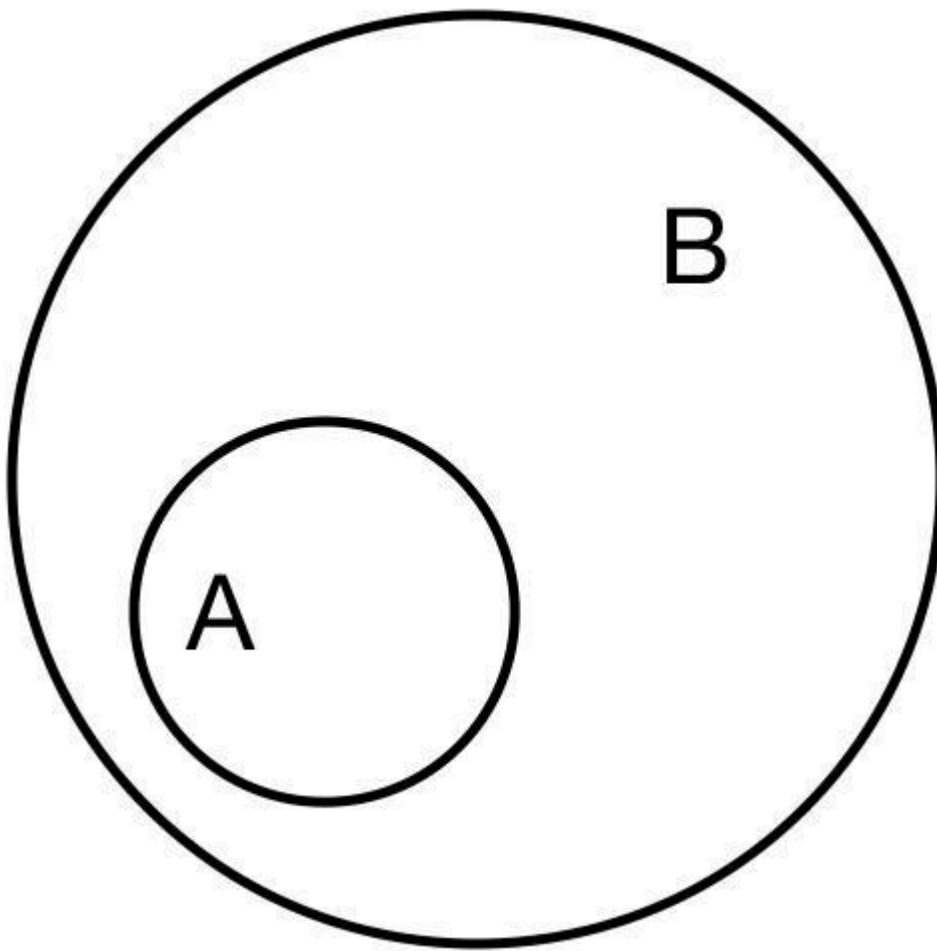
If Ali belongs to the programmers set, we can conclude that Ali belongs to the problem-solvers set.

This diagram helps visualize the reasoning behind a valid argument.

However, diagrams must be interpreted carefully. A picture can illustrate a logical relationship, but the validity of an argument still depends on the formal statements and the assumptions represented.

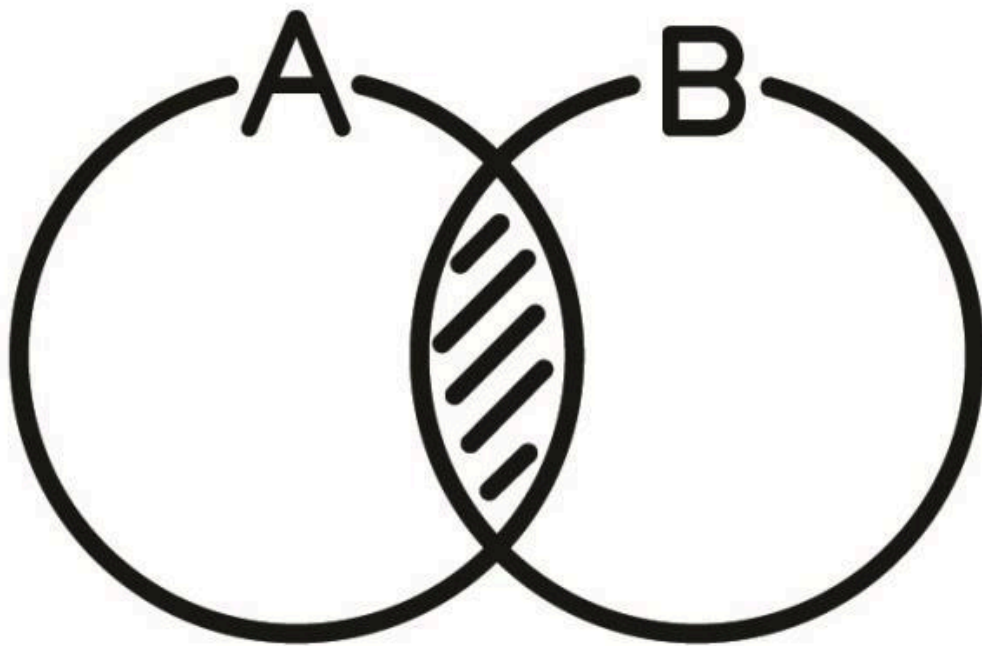
7.6 Euler diagrams vs. Venn diagrams

These two diagrams look similar but serve slightly different purposes.



Euler diagram

Shows only the relationships that actually hold or are asserted between sets.



Venn diagram

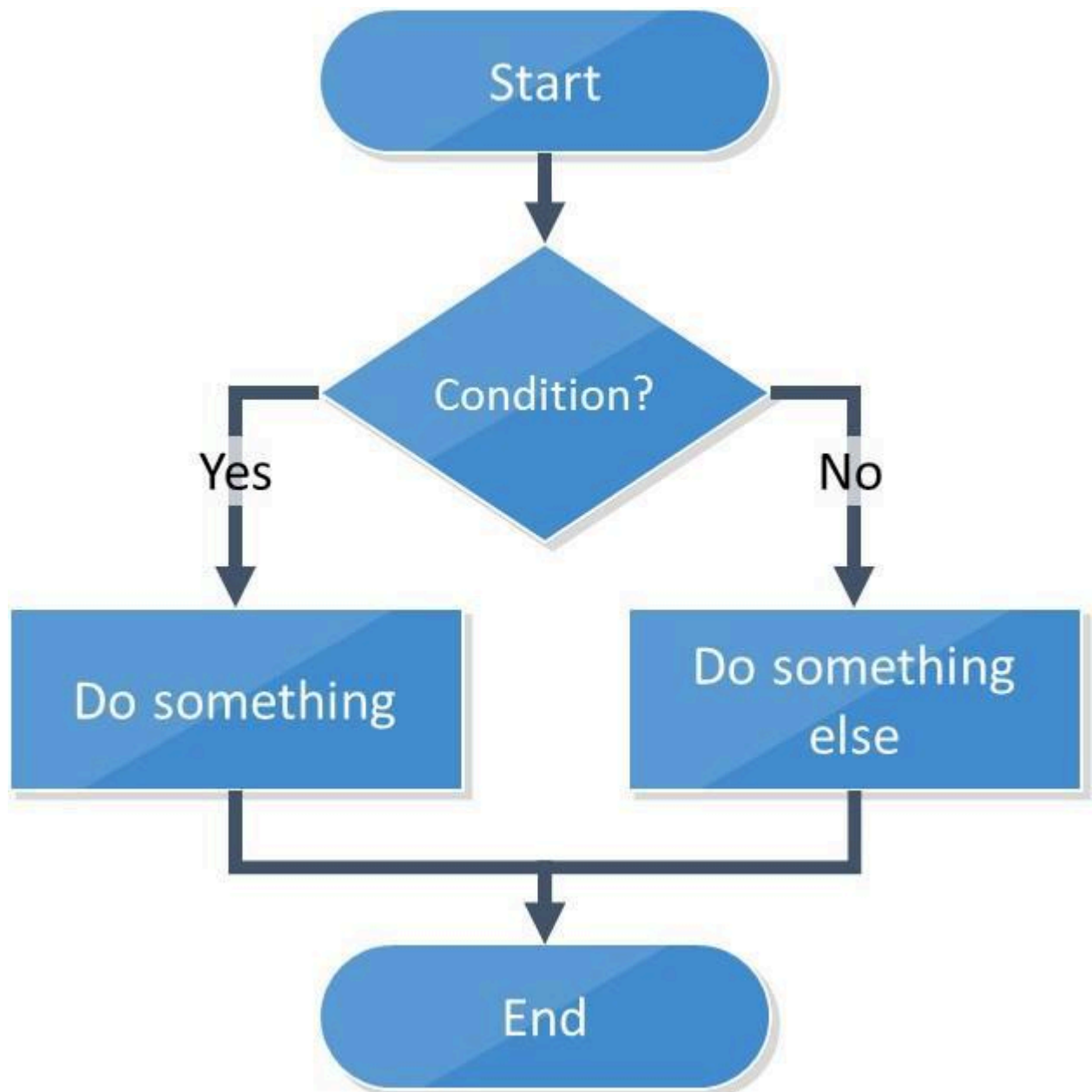
Shows all possible intersections between sets, including regions that may be empty.

For example, if every cat is a mammal, an Euler diagram can represent the cat circle entirely inside the mammal circle.

A Venn diagram for the two categories displays all possible regions, even if some of those regions are empty in the situation being considered.

7.7 Other types of diagrams used in computer science

Diagrammatic logic is broader than Venn diagrams.

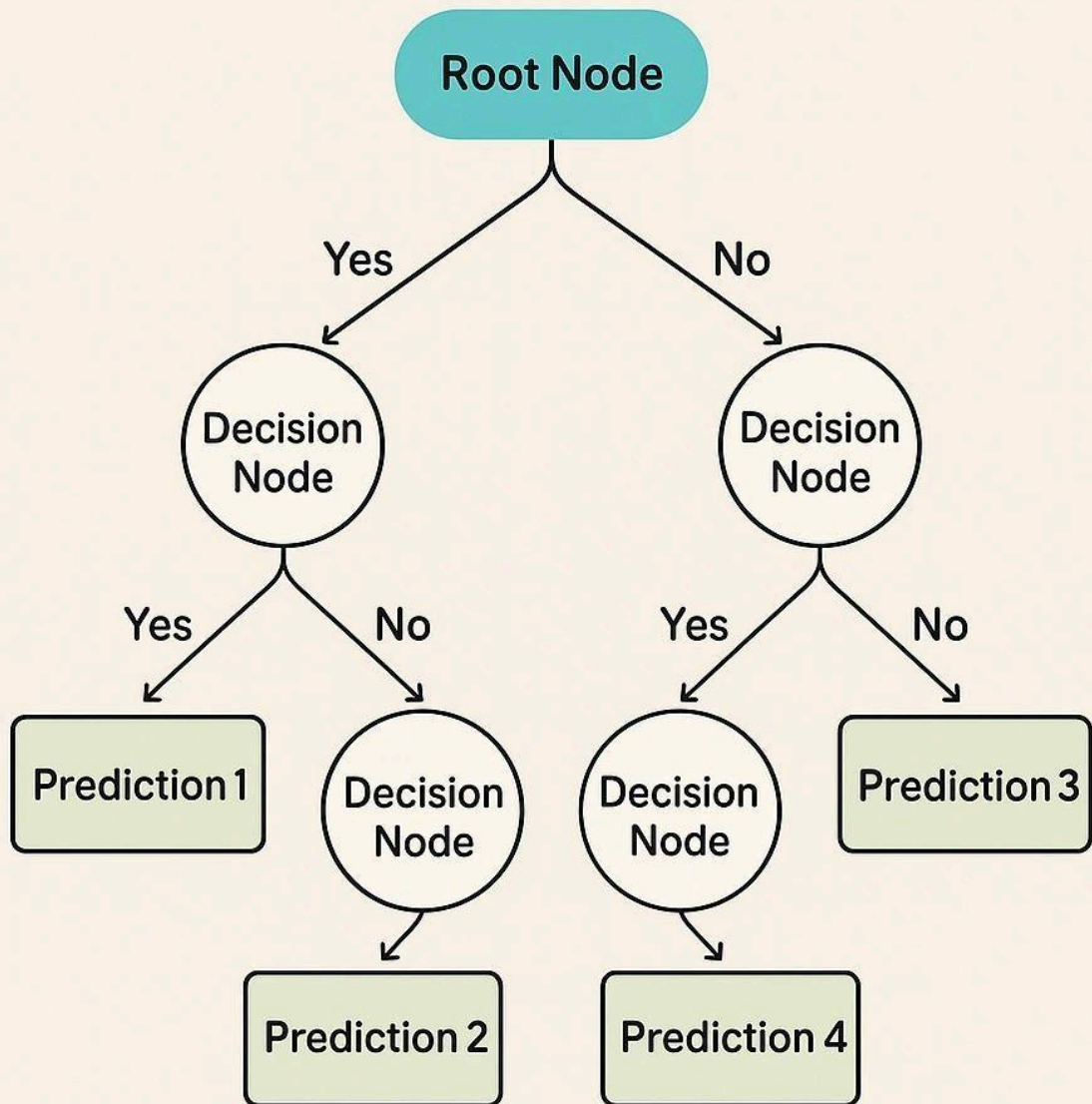


1. Flowcharts

Flowcharts represent the steps and decisions of an algorithm visually.

Example: A flowchart for checking whether a number is even or odd uses a decision diamond to test whether the remainder after division by 2 is zero.

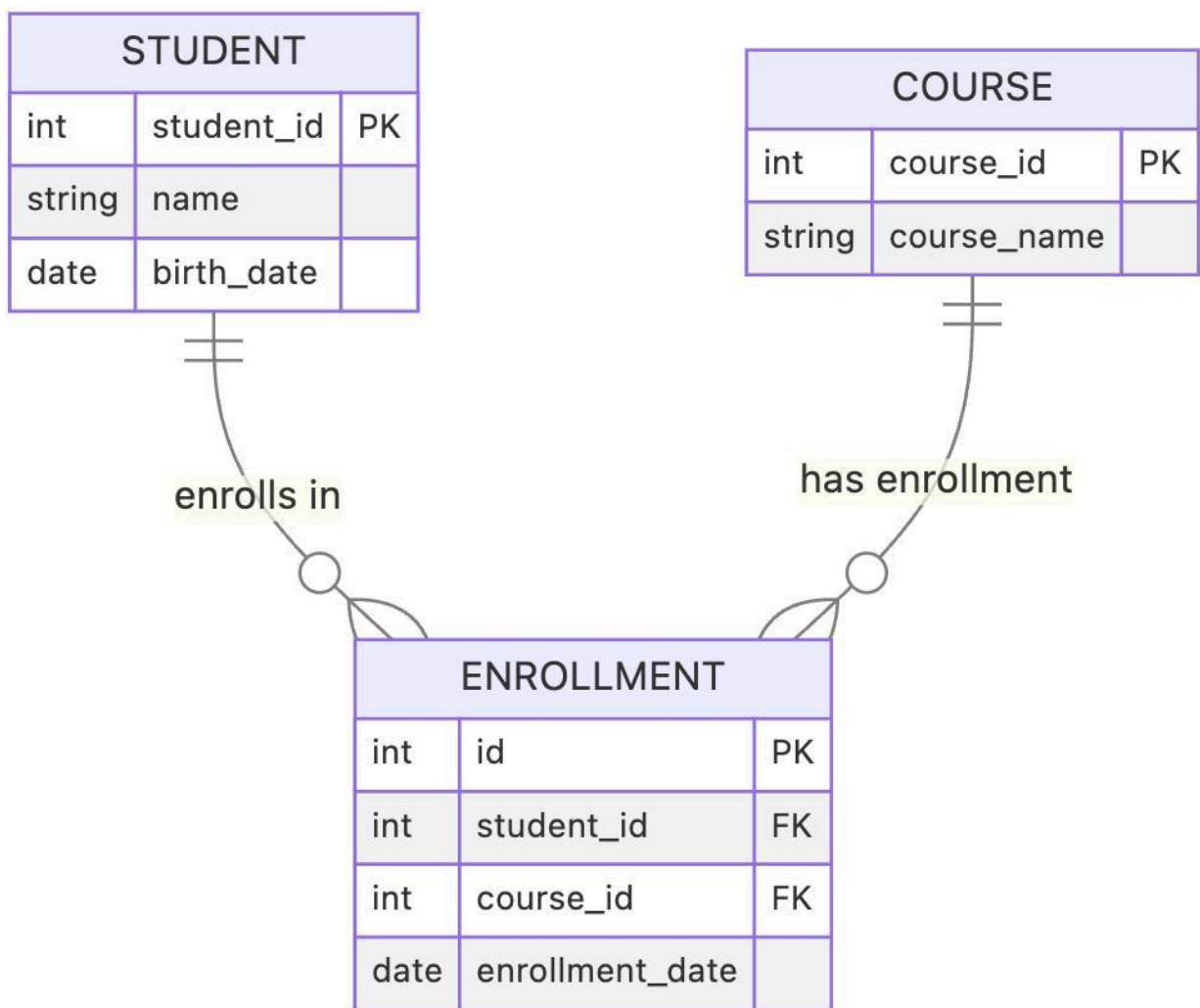
Decision Tree Model



2. Decision trees

Decision trees represent a series of decisions and their possible outcomes.

They are used in algorithms, machine learning, and classification problems.



3. Relationship diagrams

These represent connections between entities or objects.

They are useful in database design, data modeling, and system analysis.

Lecture 8: Sources of Illogical Thinking

1. What is illogical thinking?

Illogical thinking is a way of reasoning in which conclusions are not properly supported by evidence, facts, or valid reasoning.

Example:

- Every programmer I know uses Python.
- Therefore, every programmer in the world uses Python.

The conclusion is not justified because your experience is limited.

Remember: A conclusion can accidentally be correct even if the reasoning used to reach it is poor.

2. Major sources of illogical thinking

1. Cognitive biases

Cognitive biases are systematic patterns of thinking that can lead us to make errors in judgment.

Example:

You believe that online learning is ineffective. You focus on stories of students who failed online courses while ignoring students who succeeded.

Your existing belief affects how you evaluate information.

2. Confirmation bias

Confirmation bias is the tendency to seek, interpret, and remember information that supports our existing beliefs while ignoring contradictory evidence.

Example:

You think a particular programming language is the fastest. You only read articles praising its speed and ignore benchmarks showing situations where other languages perform better.

How to avoid it: Actively look for evidence that could prove your belief wrong.

3. Availability heuristic

The availability heuristic occurs when we judge how common or likely something is based on how easily examples come to mind.

Example:

You see several news reports about cyberattacks. You conclude that every computer system is constantly being hacked.

Recent or memorable events may feel more common than they actually are.

4. Anchoring bias

Anchoring is the tendency to rely too heavily on the first piece of information received when making a decision.

Example:

A laptop is initially advertised for Rs. 200,000 and then discounted to Rs. 150,000.

You may think it's a great deal simply because the original price influenced your judgment, even if the laptop's actual market value is Rs. 140,000.

5. Emotional reasoning

Emotional reasoning occurs when someone treats their feelings as proof that a conclusion is true.

Example:

"I feel that this difficult programming assignment is impossible. Therefore, I cannot solve it."

Feeling incapable does not prove that you lack the ability to solve the problem.

Important: Emotions are real experiences, but they are not always reliable evidence about external facts.

6. Overgeneralization

Overgeneralization means drawing a broad conclusion from a small or limited number of examples.

Example:

- One student cheated in an examination.
- Therefore, all students cheat.

The evidence supports a conclusion about one student, not every student.

7. Assumptions

An assumption is something accepted as true without sufficient proof.

Assumptions can be useful when solving problems, but untested assumptions can produce incorrect conclusions.

Example:

You assume that a website is secure because it has a professional design.

A website's appearance does not establish that its database, authentication system, or network communication is secure.

8. Social influence and peer pressure

Sometimes people accept an idea because many others believe it, rather than because the idea has been supported by evidence.

Example:

Your entire class believes that a particular exam question has only one correct answer. You accept that belief without checking the actual question.

A popular belief is not automatically a correct belief.

9. Ambiguous language

Ambiguous language occurs when words or statements have multiple possible meanings.

Example:

"Students who study hard pass."

Does this mean every student who studies hard passes? Or that studying hard increases the likelihood of passing?

The statement is unclear without more precise wording.

In programming, ambiguity can also cause misunderstandings about requirements.

10. Inadequate evidence

A conclusion may be unreliable when the available evidence is insufficient, incomplete, or unverified.

Example:

You test a program with only two inputs. Both produce correct results, so you declare the program completely correct.

Two successful tests are not enough to prove that a program works in every possible situation.

11. Faulty memory

Human memory is not a perfect recording system. People may forget details, mix up events, or unintentionally change their recollections.

Example:

You remember that your teacher said an assignment was due on Friday, but the actual deadline was Thursday.

If you rely only on memory instead of checking the announcement, your conclusion may be wrong.

12. Lack of knowledge

Sometimes illogical conclusions arise because we do not understand the subject well enough.

Example:

A beginner sees a complex error message and concludes that Python itself is broken.

Without understanding the code, the error, or the environment, that conclusion is premature.

Lack of knowledge does not automatically make a person irrational, but it increases the risk of mistaken reasoning.

3. Illogical thinking vs. logical fallacies

These concepts are related, but they are not identical.

Illogical thinking	Logical fallacy
A broad category of reasoning mistakes and judgment errors	A recognizable error in an argument
Can arise from emotions, biases, assumptions, or limited knowledge	Often involves a specific faulty argument structure or persuasive technique
May happen without making an explicit argument	Usually identifiable in the reasoning or argument presented
Example: believing a program works after one test	Example: claiming a program is reliable because a famous programmer said so

A cognitive bias can lead someone to commit a logical fallacy, but not every bias is itself a fallacy.

4. How to prevent illogical thinking

As a computer science student, you can apply a systematic approach:

1. Identify the claim
What exactly are you trying to prove or decide?
2. Examine the evidence
What facts, observations, or tests support the claim?
3. Question your assumptions
What are you accepting without verification?
4. Consider alternatives
Could another explanation account for the same evidence?
5. Test your conclusion
Use experiments, examples, counterexamples, or reliable sources.

6. Revise your belief if necessary

Good reasoning means being willing to change your conclusion when new evidence appears.

This process is closely related to debugging. Instead of assuming your code is correct, you test it, inspect its behavior, investigate errors, and update your understanding.

Lecture 9: First-Order Logic vs. Higher-Order Logic

1. What is formal logic?

Formal logic is a system of reasoning that uses precise symbols and rules to determine whether conclusions follow from premises.

Consider this argument:

- All humans are mortal.
- Ali is a human.
- Therefore, Ali is mortal.

This is an example of a logically valid argument.

In formal logic, we can represent it symbolically:

- $H(x)$: x is human.
- $M(x)$: x is mortal.
- a : Ali.

Then:

$\forall x (H(x) \rightarrow M(x))$
 $H(a)$
 $\therefore M(a)$

Here, $\forall$ means "for all," and $\rightarrow$ means "implies."

This is the foundation for understanding first-order logic.

2. First-order logic (FOL)

First-order logic is a formal system in which we can talk about individual objects and quantify over them.

An object could be:

- A student
- A computer
- A number
- A programmer
- A database record

Main components of FOL

Component	Meaning	Example
Constants	Specific objects	Ali, 5, Computer1
Variables	Objects that may vary	x,y,zx,y,zx,y,z
Predicates	Properties or relationships	Student(x)Student(x)Student(x)
Quantifiers	Specify how many objects	$\forall, \exists$ forall, exists $\forall, \exists$
Logical connectives	Combine statements	$\wedge, \vee, \neg, \rightarrow$ and, or, neg, rightarrow $\wedge, \vee, \neg, \rightarrow$
Functions	Map objects to objects	fatherOf(x)fatherOf(x)fatherOf(x)

Two important quantifiers

Universal quantifier ($\forall$ forall $\forall$)

Means "for every" or "for all."

Example:

$\forall x(\text{Student}(x) \rightarrow \text{Learner}(x))$ forall x(Student(x) $\rightarrow$ Learner(x)) $\forall x(\text{Student}(x) \rightarrow \text{Learner}(x))$

Meaning: Every student is a learner.

Existential quantifier ($\exists$ exists $\exists$)

Means "there exists at least one."

Example:

$\exists x(\text{Student}(x) \wedge \text{Programmer}(x))$ exists x(Student(x) and Programmer(x)) $\exists x(\text{Student}(x) \wedge \text{Programmer}(x))$

Meaning: At least one student is a programmer.

Real-world computer science example

Suppose a university system stores student records.

We define:

- $\text{Student}(x)$: x is a student.
- $\text{Enrolled}(x, \text{CS101})$: x is enrolled in the CS101 course.

To say that every student is enrolled in CS101:

$$\forall x (\text{Student}(x) \rightarrow \text{Enrolled}(x, \text{CS101}))$$

To say that at least one student is enrolled in CS101:

$$\exists x (\text{Student}(x) \wedge \text{Enrolled}(x, \text{CS101}))$$

FOL lets us express these relationships precisely.

3. Higher-order logic (HOL)

Higher-order logic extends first-order logic by allowing quantification over things such as predicates, properties, relations, and functions.

In FOL, we can say:

"Every student is a learner."

$$\forall x (\text{Student}(x) \rightarrow \text{Learner}(x))$$

Here, we quantify over individual objects x .

In HOL, we can also express statements about properties themselves.

For example:

"Every property that holds for all objects also holds for Ali."

$$\forall P ((\forall x P(x)) \rightarrow P(\text{Ali}))$$

Here, P represents a property, and we quantify over possible properties rather than only individual objects.

This is the key difference: HOL lets us quantify over predicates and other higher-level entities.

A simple programming example

Imagine you have several functions in Python:

```
def square(x):  
    return x * x
```

```
def double(x):  
    return x * 2
```

```
def increment(x):  
    return x + 1
```

You can write a function that accepts another function as an argument:

```
def apply_operation(operation, number):  
    return operation(number)
```

```
print(apply_operation(square, 5))
```

Output:

25

Here, `operation` is itself a function.

Higher-order logic can reason formally about functions and properties as objects of quantification. The Python example illustrates the idea of treating functions as values; it is not itself a formal HOL expression.

4. Key differences between FOL and HOL

Feature	First-order logic	Higher-order logic
Main focus	Individual objects	Objects, properties, relations, and functions
Quantification	Over individual objects	Can also quantify over predicates and functions
Expressive power	More restricted	More expressive
Complexity	Generally simpler	Often more complex
Example	Every student studies	Every property shared by all objects holds for a given object

Applications

Databases, knowledge
representation, mathematics

Theorem proving, mathematical
foundations, formal verification

One important detail: higher-order logic has different formal systems. Their exact expressive power and computational properties depend on the system being used.

5. Understanding the difference through an analogy

Imagine a university.

First-order logic: You talk about students.

- Student A is intelligent.
- Student B is hardworking.
- Every student has an ID.

You are reasoning about individual students and their relationships or properties.

Higher-order logic: You talk about properties of students.

- Intelligence is a property.
- Hardworking is a property.
- You can reason about properties that all students share.
- You can quantify over a collection of properties.

You are now reasoning not only about students but also about the properties themselves.

Remember this distinction

- First-order: "For every student xxx, ..."
- Higher-order: "For every property PPP, ..."

6. Applications in computer science



Databases and knowledge representation

First-order logic is closely related to relational database concepts. It can represent facts and relationships, such as which student is enrolled in which course.

```
section < Euclidean Algorithm in Isabelle/UTP >

theory Euclidean_Algorithm [1 lines]
begin

zstore gcd_st =
  a :: nat
  b :: nat

program eucl "(A::nat, B::nat)" over gcd_st =
"a := A; b := B;
while a ≠ b
  invariant a > 0 ∧ b > 0 ∧ gcd a b = gcd A B
  variant a + b
do
  if a > b then a := a - b else b := b - a
fi
od"

lemma eucl_correct: "H[A > 0 ∧ B > 0] eucl (A, B) [a = gcd A B]"
  apply vcg
  apply (metis gcd_diff1_nat linorder_not_less not_less_iff_gr_or_eq)
  apply (metis gcd.commute gcd_diff1_nat linorder_not_less)
done

end
```

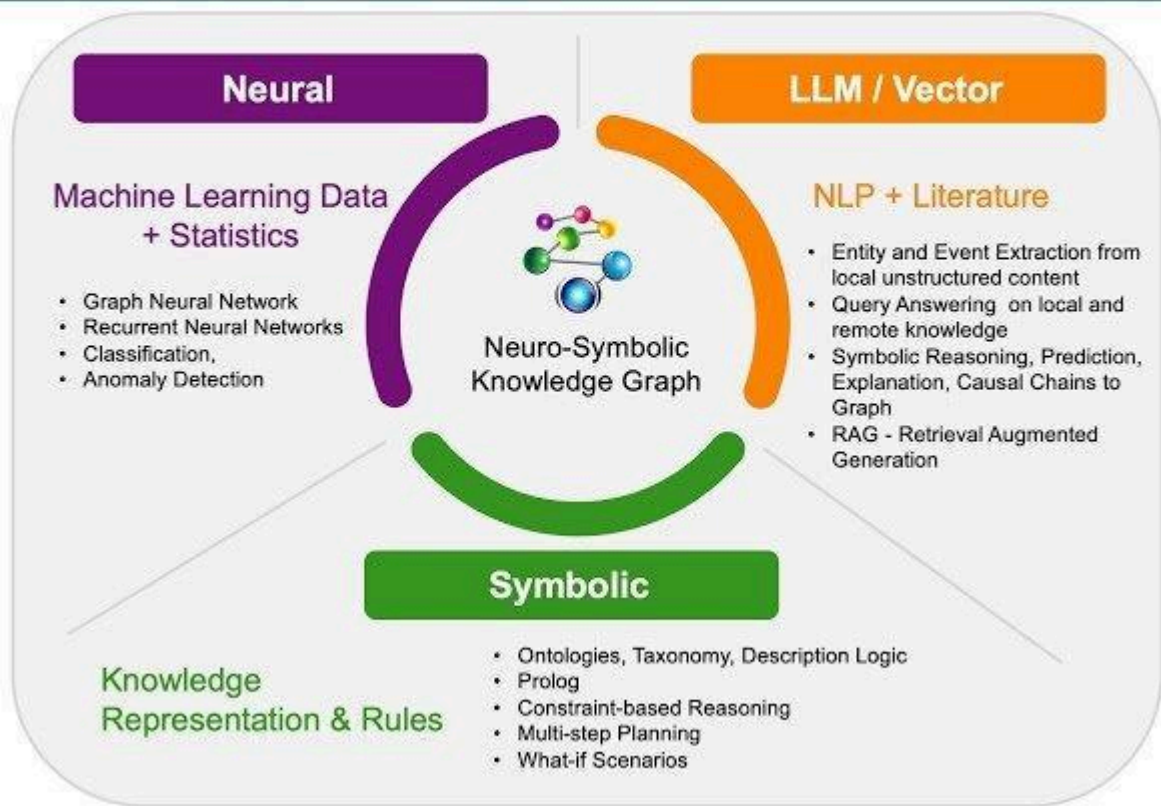
26,1 (576/580) (isabelle,isabelle,UTF-8-Isabelle) | n m r o U G JVM: 302/512MiB ML: 409/924MiB 10:52

Formal verification

Logic is used to specify and prove properties of software and hardware. Higher-order logic is used in some theorem-proving systems to express sophisticated mathematical and program properties.

Neuro-Symbolic AI

Knowledge Graph Architecture



Artificial intelligence

First-order logic can represent facts and rules for reasoning systems. Higher-order formalisms can represent more complex relationships and abstract properties, although many practical AI systems use other representations.